

恶意代码分析与防治技术实验报告

Lab5-1



网络空间安全学院

信息安全专业

2211985 李佳璐

一、实验目的

1. 熟悉IDAPro的恶意代码分析过程

二、实验原理

IDA Pro (Interactive DisAssembler Pro) 是一款功能强大的反汇编和逆向工程工具，广泛应用于安全研究、恶意软件分析和程序调试。以下是IDA Pro的一些主要特点：

- ### 三、实验过程

```
.text:1000002E
.text:1000002E
.text:1000002E ; BOOL __stdcall DllMain(HINSTANCE hinstDLL, DWORD FdwReason, LPVOID lpvReserved)
.text:1000002E _DllMain@12 proc near ; CODE XREF: DllEntryPoint+4B↓p
.text:1000002E ; DATA XREF: sub_100110FF+2D↓p
.text:1000002E
.text:1000002E hinstDLL = dword ptr 4
.text:1000002E FdwReason = dword ptr 8
.text:1000002E lpvReserved = dword ptr 0Ch
.text:1000002E
```

在Imports窗口使用Search搜索gethostbyname

Address	Ordinal	Name	Library
000000...		ReleaseDC	USER32
000000...		OpenInputDesktop	USER32
000000...		GetUserObjectInformationA	USER32
000000...		waveInReset	WINMM
000000...		waveInOpen	WINMM
000000...		waveInClose	WINMM
000000...		waveInUnprepareHeader	WINMM
000000...		waveInPrepareHeader	WINMM
000000...		waveInAddBuffer	WINMM
000000...		waveInStart	WINMM
000000...	18	select	WS2_32
000000...	11	inet_addr	WS2_32
000000...	52	gethostbyname	WS2_32
000000...	12	inet_ntoa	WS2_32
000000...	16	recv	WS2_32
000000...	19	send	WS2_32
000000...	4	connect	WS2_32
000000...	15	ntohs	WS2_32
000000...	9	htons	WS2_32
000000...	21	setsockopt	WS2_32
000000...	116	WSACleanup	WS2_32
000000...	115	WSAStartup	WS2_32
000000...	3	closesocket	WS2_32
000000...	23	socket	WS2_32
000000...	111	WSAGetLastError	WS2_32

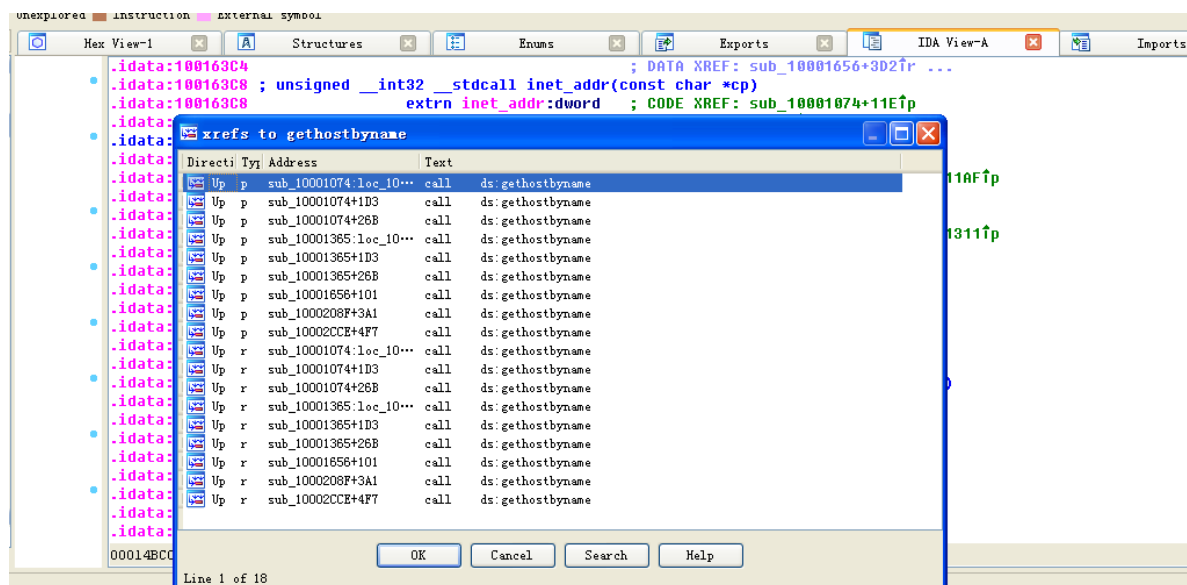
双击跳转

[illegible]

可以看到其地址为 `.idata 0x100163CC`。

3. 有多少函数调用了 `gethostbyname`?

在当前 `gethostbyname` 使用 `CTRL + X` 查看其交叉引用，可以看到有18行记录。



可以看到IDAPro将p（被调用的引用）与r（被读取的引用）都予以计算，故而是9次引用。仔细查看交叉引用列表，通过Address中的 `sub_函数入口地址` 可以看到共有5个不同的函数调用了该函数。

4. 将精力集中在位于0x1001757处的对 `gethostbyname` 的调用，你能找出哪个DNS请求将被触发吗？

`Up r sub_10001656+101 call ds: gethostbyname`

通过双击该条记录，即可跳转到

```
.text:10001737      add     esp, 0Ch
.text:1000173A      test    eax, eax
.text:1000173C      jz      loc_100017ED
.text:10001742      cmp     dword_1008E5CC, ebx
.text:10001748      jnz     loc_100017ED
.text:1000174E      mov     eax, off_10019040
.text:10001753      add     eax, 0Dh
.text:10001756      push    eax ; name
.text:10001757      call    ds: gethostbyname
```

同时可以看到在寄存器 `eax` 中加了 `0Dh` 字节的偏移

继续双击跳转

```
.data:10019040 off_10019040 dd offset aThisIsRdoPics
.data:10019040 ; DATA XREF: sub_10001656:loc_10001722↑r
.data:10019040 ; sub_10001656+F8↑r ...
.data:10019040 ; "[This is RDO]pics.practicalmalwareanalysis"...
```

可以看到 `gethostbyname` 使用的 `EAX` 是一个指向 `pics.practicalmalwareanalysis.com` 的指针。在 `eax` 基础上加 `0xD` 偏移后得到URL位置，内存中字符串的存储内容如上图。这个调用会对该域名发起一个DNS请求，以获得其IP地址。

5. IDA Pro识别了在0x10001656处的子过程中的多少个局部变量？

按下G键到 `0x10001656` 处，分析 `sub_10001656` 的代码。

```

.text:10001656 ; DWORD __stdcall sub_10001656(LPVOID lpThreadParameter)
.text:10001656 sub_10001656 proc near ; DATA XREF: DllMain(x,x,x)+C8↓o
.text:10001656
.text:10001656 var_675 = byte ptr -675h
.text:10001656 var_674 = dword ptr -674h
.text:10001656 hModule = dword ptr -670h
.text:10001656 timeout = timeval ptr -66Ch
.text:10001656 name = sockaddr ptr -664h
.text:10001656 var_654 = word ptr -654h
.text:10001656 Dst = dword ptr -650h
.text:10001656 Str1 = byte ptr -644h
.text:10001656 var_640 = byte ptr -640h
.text:10001656 CommandLine = byte ptr -63Fh
.text:10001656 Str = byte ptr -63Dh
.text:10001656 var_638 = byte ptr -638h
.text:10001656 var_637 = byte ptr -637h
.text:10001656 var_544 = byte ptr -544h
.text:10001656 var_50C = dword ptr -50Ch
.text:10001656 var_500 = byte ptr -500h
.text:10001656 Buf2 = byte ptr -4FCh
.text:10001656 readfds = fd_set ptr -48Ch
.text:10001656 buf = byte ptr -3B8h
.text:10001656 var_3B0 = dword ptr -3B0h
.text:10001656 var_1A4 = dword ptr -1A4h
.text:10001656 var_194 = dword ptr -194h
.text:10001656 WSAData = WSAData ptr -190h
.text:10001656 lpThreadParameter = dword ptr 4

```

IDA Pro已经完成对函数局部变量和参数的识别，并做了标记。被标记的局部变量与负偏移值相关，一共有23个，其中大部分前缀是var_。

6. IDA Pro 识别了在0x10001656处的子过程中的多少个参数？

```

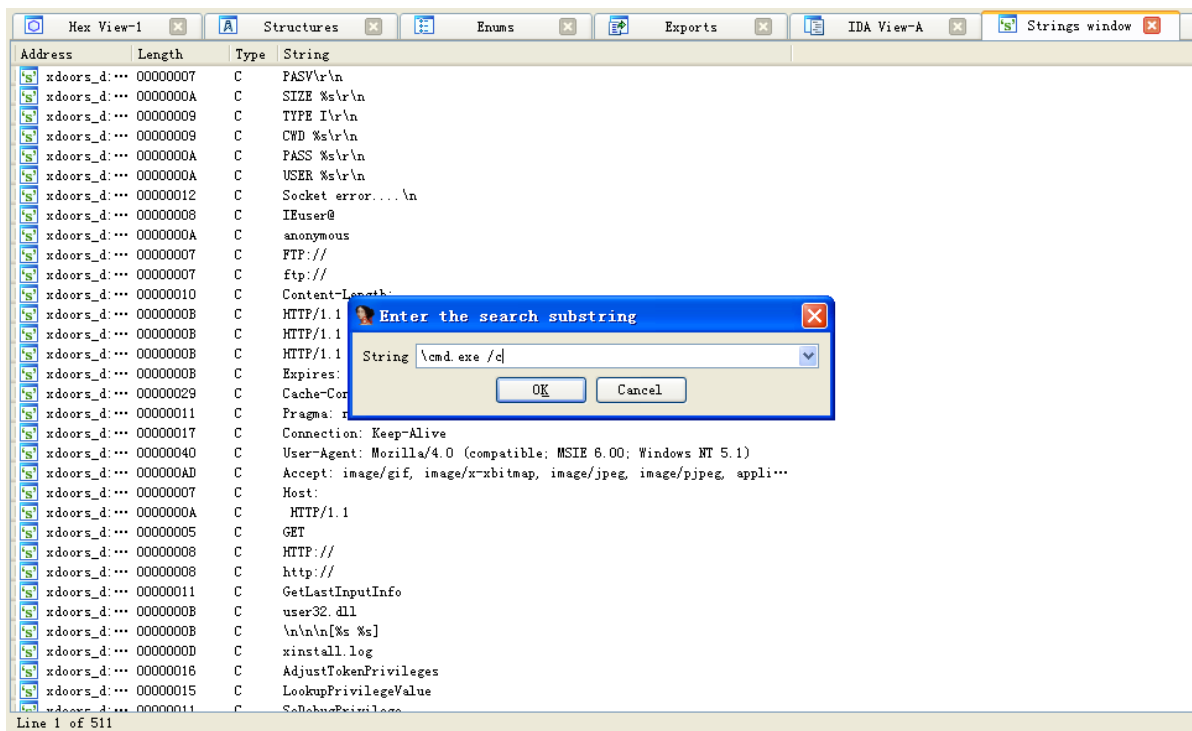
.text:10001656 ; DWORD __stdcall sub_10001656(LPVOID lpThreadParameter)
.text:10001656 sub_10001656 proc near ; DATA XREF: DllMain(x,x,x)+C8↓o
.text:10001656
.text:10001656 var_675 = byte ptr -675h
.text:10001656 var_674 = dword ptr -674h
.text:10001656 hModule = dword ptr -670h
.text:10001656 timeout = timeval ptr -66Ch
.text:10001656 name = sockaddr ptr -664h
.text:10001656 var_654 = word ptr -654h
.text:10001656 Dst = dword ptr -650h
.text:10001656 Str1 = byte ptr -644h
.text:10001656 var_640 = byte ptr -640h
.text:10001656 CommandLine = byte ptr -63Fh
.text:10001656 Str = byte ptr -63Dh
.text:10001656 var_638 = byte ptr -638h
.text:10001656 var_637 = byte ptr -637h
.text:10001656 var_544 = byte ptr -544h
.text:10001656 var_50C = dword ptr -50Ch
.text:10001656 var_500 = byte ptr -500h
.text:10001656 Buf2 = byte ptr -4FCh
.text:10001656 readfds = fd_set ptr -48Ch
.text:10001656 buf = byte ptr -3B8h
.text:10001656 var_3B0 = dword ptr -3B0h
.text:10001656 var_1A4 = dword ptr -1A4h
.text:10001656 var_194 = dword ptr -194h
.text:10001656 WSAData = WSAData ptr -190h
.text:10001656 lpThreadParameter = dword ptr 4

```

参数被标记和引用为正的偏移值，可以看到识别了子过程中的一个参数 lpThreadParameter

7. 使用Strings窗口，在反汇编中定位字符串cmd.exe /c。它位于哪？

通过菜单 View→OpenSubviews→Strings，来查看这个DLL的字符串。



```

xdoors_d:00000011 C \\command.exe /c
xdoors_d:0000000D C \\cmd.exe /c
xdoors_d:00000118 C Hi,Master [%d/%d/%d %d:%d:%d]\r\nWelCome Back...Are You Enjoying T...

```

双击跳转到反汇编窗口

```

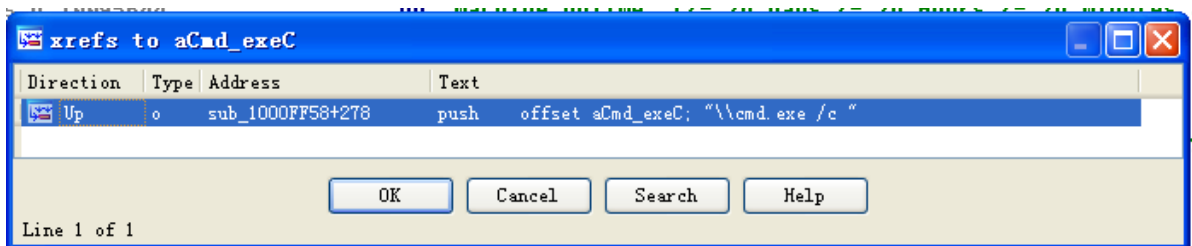
xdoors_d:10095B34 aCmd_exeC db '\\cmd.exe /c' ; DATA XREF: sub_1000FF58+278f0 |
xdoors_d:10095B41 align 4
xdoors_d:10095B44 ; char aHiMasterDDDDDD[]
xdoors_d:10095B44 aHiMasterDDDDDD db 'Hi,Master [%d/%d/%d %d:%d:%d]',0Dh,0Ah
xdoors_d:10095B44 ; DATA XREF: sub_1000FF58+145f0
xdoors_d:10095B44 db 'WelCome Back...Are You Enjoying Today?',0Dh,0Ah
xdoors_d:10095B44 db 0Dh,0Ah
xdoors_d:10095B44 db 'Machine UpTime [%-.2d Days %-.2d Hours %-.2d Minutes %-.2d Secon'
xdoors_d:10095B44 db 'ds]',0Dh,0Ah
xdoors_d:10095B44 db 'Machine IdleTime [%-.2d Days %-.2d Hours %-.2d Minutes %-.2d Seco'
xdoors_d:10095B44 db 'nds]',0Dh,0Ah
xdoors_d:10095B44 db 0Dh,0Ah
xdoors_d:10095B44 db 'Encrypt Magic Number For This Remote Shell Session [0x%02x]',0Dh,0Ah
xdoors_d:10095B44 db 0Dh,0Ah,0
xdoors_d:10095C5C ; char asc_10095C5C[]
xdoors_d:10095C5C asc_10095C5C db '>',0 ; DATA XREF: sub_1000FF58+4Bf0
xdoors_d:10095C5C ; sub_1000FF58+3E1f0
xdoors_d:10095C5E align 400h
xdoors_d:10095C5E xdoors_d ends
xdoors_d:10095C5E
xdoors_d:10095C5E end DllEntryPoint

```

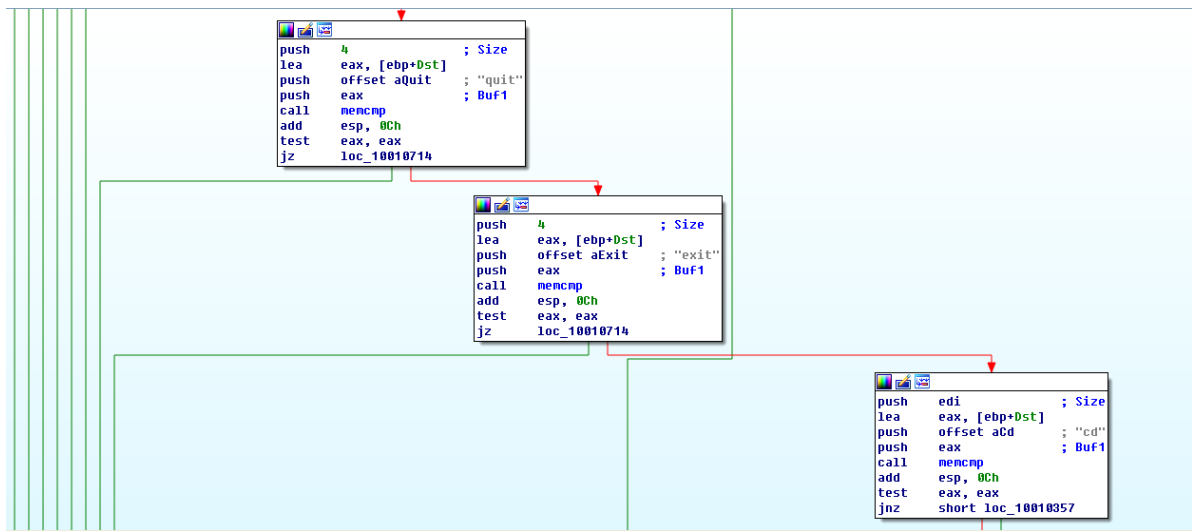
该字符串位于该PE文件的xdoors_d节中的0x10095B34处。

8. 在引用\\cmd.exe /c 的代码所在区域发生了什么？

查看这个字符串的交叉引用



可以看到该字符串被压到栈上；



从这个函数的图形模式，可以看到一系列的memcmp函数被用于比较如cd、exit、quit、install、inject和uptime等的字符串。

在字符串被引用之前，可以看到字符串“Hi,Master.....”

```

push    eax
movzx   eax, [ebp+SystemTime.wMonth]
push    eax
movzx   eax, [ebp+SystemTime.wYear]
push    eax
lea     eax, [ebp+Dst]
push    offset aHiMasterDDDDDD ; "Hi,Master [%d/%d/%d %d:%d:%d]\r\nWelCom"...
push    eax ; Dest
call    ds:sprintf
add     esp, 44h
xor     ebx, ebx
lea     eax, [ebp+Dst]
push    ebx
push    eax ; Str
call    strlen
pop     ecx
push    eax ; len
lea     eax, [ebp+Dst]

```

查看一下这个函数和它发起的调用，可以看到有一系列的recv、send、connect、closesocket等调用。

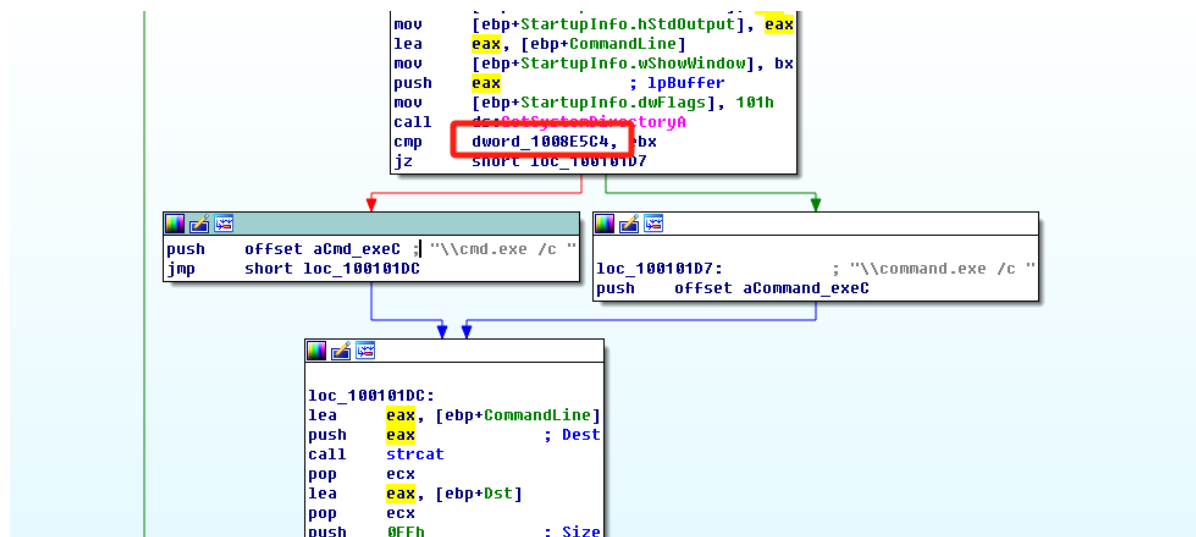
```

.text:10010782      call     ds:ntohs
.text:10010788      mov     word ptr [ebp+name.sa_data], ax
.text:1001078C      lea     eax, [ebp+name]
.text:1001078F      push    10h          ; namelen
.text:10010791      push    eax          ; name
.text:10010792      push    esi          ; s
.text:10010793      call    ds:connect
.text:10010799      cmp     eax, 0FFFFFFFh
.text:1001079C      jz      short loc_100107E1
.text:1001079E      mov     [ebp+var_22C], 7
.text:100107A8      call    ds:GetTickCount
.text:100107AE      mov     [ebp+var_230], eax
.text:100107B4      and     eax, 1Fh
.text:100107B7      inc     eax
.text:100107B8      push    0            ; flags
.text:100107BA      mov     [ebp+var_24], eax
.text:100107BD      mov     dword_1008E5D0, eax
.text:100107C2      lea     eax, [ebp+buf]
.text:100107C8      push    228h          ; len
.text:100107CD      push    eax          ; buf
.text:100107CE      push    esi          ; s
.text:100107CF      call    ds:send
.text:100107D5      cmp     eax, 0FFFFFFFh
.text:100107D8      jz      short loc_100107E1
.text:100107DA      push    esi          ; s
.text:100107DB      call    sub_1000FF58
.text:100107E0      pop     ecx
.text:100107E1      loc_100107E1:
.text:100107E1      ; CODE XREF: StartAddress+1B↑j
.text:100107E1      ; StartAddress+5C↑j ...
.text:100107E1      push    esi          ; s
.text:100107E2      call    ds:closesocket
.text:100107E8      xor     eax, eax
.text:100107EA      pop     esi

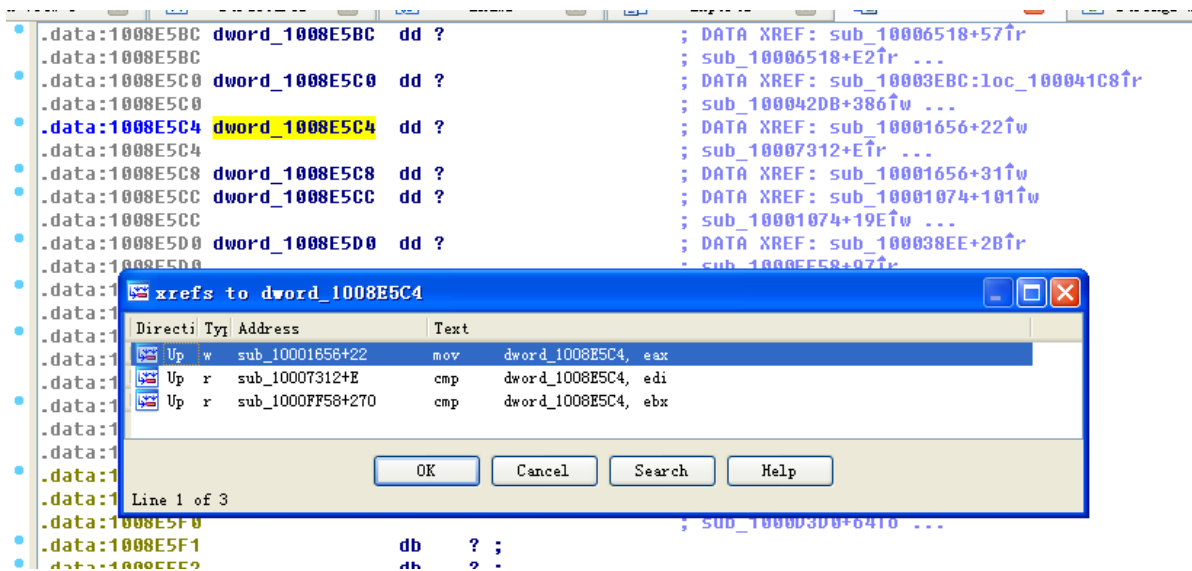
```

根据以上线索，可以猜测正在查看的是一个远程shell会话函数。

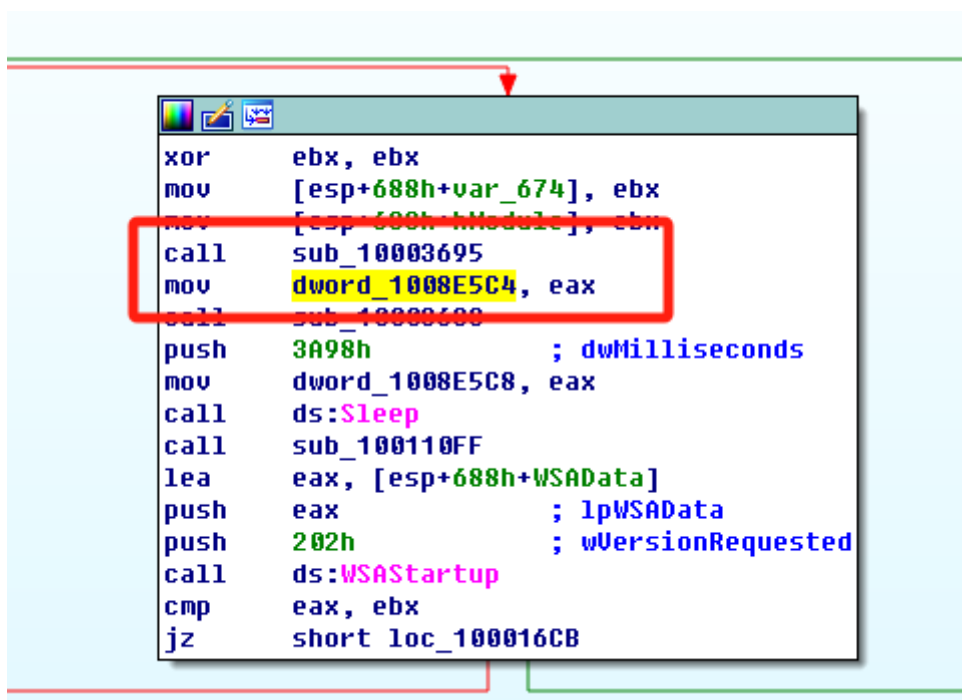
9. 在同样的区域，在0x100101C8处，看起来好像dword_1008E5C4是一个全局变量，它帮助决定走哪条路径。那恶意代码是如何设置dword_1008E5C4的呢？（提示：使用dword_1008E5C4的交叉引用）



双击来到dll文件的.data节，使用CTRL+X查看其交叉引用



可以看到dword_1008E5C4被引用了三次，仅有一个引用进行的是写类型的，对dword_1008E5C4进行了修改。



可以看到，EAX被赋给dword_1008E5C4，而EAX是前一条指令函数调用的返回值。

双击sub_18803695在反汇编窗口中进行查看：


```

; Attributes: bp-based frame

sub_10003695 proc near

VersionInformation= _OSVERSIONINFOA ptr -94h

push    ebp
mov     ebp, esp
sub     esp, 94h
lea     eax, [ebp+VersionInformation]
mov     [ebp+VersionInformation.dwOSVersionInfoSize], 94h
push    eax                ; lpVersionInformation
call    ds:GetVersionExA
xor     eax, eax
cmp     [ebp+VersionInformation.dwPlatformId], 2
setz    al
leave
retn
sub_10003695 endp

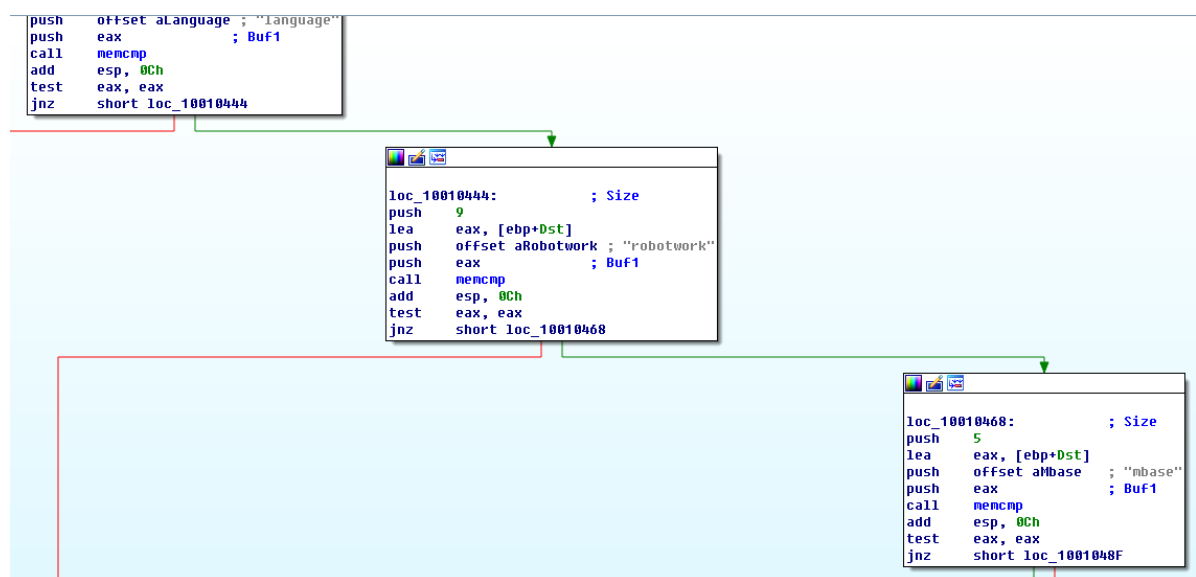
```

该函数包括了一个GetVersionEx的调用，该函数用于获取当前操作系统版本的信息。

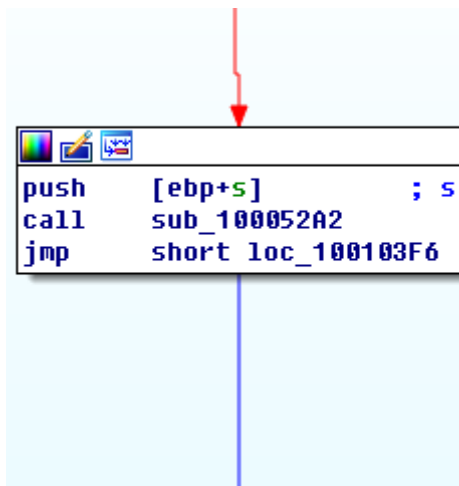
其中将 dwPlatformId 与数字2进行比较，来确定如何设置**AL寄存器**。如果 PlatformId 为 VER_PLATFORM_WIN32NT，AL会被置位。这里只是简单的判断当前操作系统是否为 Windows 2000 或更高版本，根据微软的文档，我们得知通常情况下 dwPlatformId 的值为2。那么该**全局变量通常会被置为1**。

10. 在位于0x1000FF58处的子过程中的几百行指令中，有一系列使用memcmp 来比较字符串的指令。如果对rotbotwork的字符串比较是成功的（memcmp返回0），会发生什么？

查看图形模式：



如果该字符串为robotwork，则在处的jnz不会跳转，将会调用下面这个代码块：



继续查看sub_100052A2的代码:

可以看到其参数为socket类型, 即上面push的[ebp+s]

```

; int __cdecl sub_100052A2(SOCKET s)
sub_100052A2 proc near

```

```

Dest= byte ptr -60Ch
var_60B= byte ptr -60Bh
Data= byte ptr -20Ch
var_20B= byte ptr -20Bh
cbData= dword ptr -0Ch
Type= dword ptr -8
phkResult= dword ptr -4
s= dword ptr 8

```

```

push    ebp
mov     ebp, esp
sub     esp, 60Ch
and     [ebp+Dest], 0
push    edi
mov     ecx, 0FFh
xor     eax, eax
lea     edi, [ebp+var_60B]
and     [ebp+Data], 0
rep stosd
stosw
stosb
push    7Fh
xor     eax, eax
pop     ecx
lea     edi, [ebp+var_20B]
rep stosd
stosw
stosb
lea     eax, [ebp+phkResult]

```

```

push    0F003Fh      ; samDesired
push    0             ; uIOptions
push    offset aSoftwareMicros ; "SOFTWARE\\Microsoft\\Windows\\CurrentVe..."
push    80000002h     ; hKey
call    ds:RegOpenKeyExA
test    eax, eax
jz      short loc_10005309

```

```

loc_10005309:
push    ebx
lea     eax, [ebp+cbData]
push    esi
push    eax           ; lpCbData
lea     eax, [ebp+Data]
mov     ebx, ds:RegQueryValueExA
push    eax           ; lpData
lea     eax, [ebp+Type]
push    eax           ; lpType
push    0             ; lpReserved
push    offset aWorktime ; "WorkTime"
push    [ebp+phkResult] ; hKey
call    ebx           ; RegQueryValueExA
mov     esi, ds:sprintf
mov     edi, ds:atoi
test    eax, eax
jnz     short loc_10005379

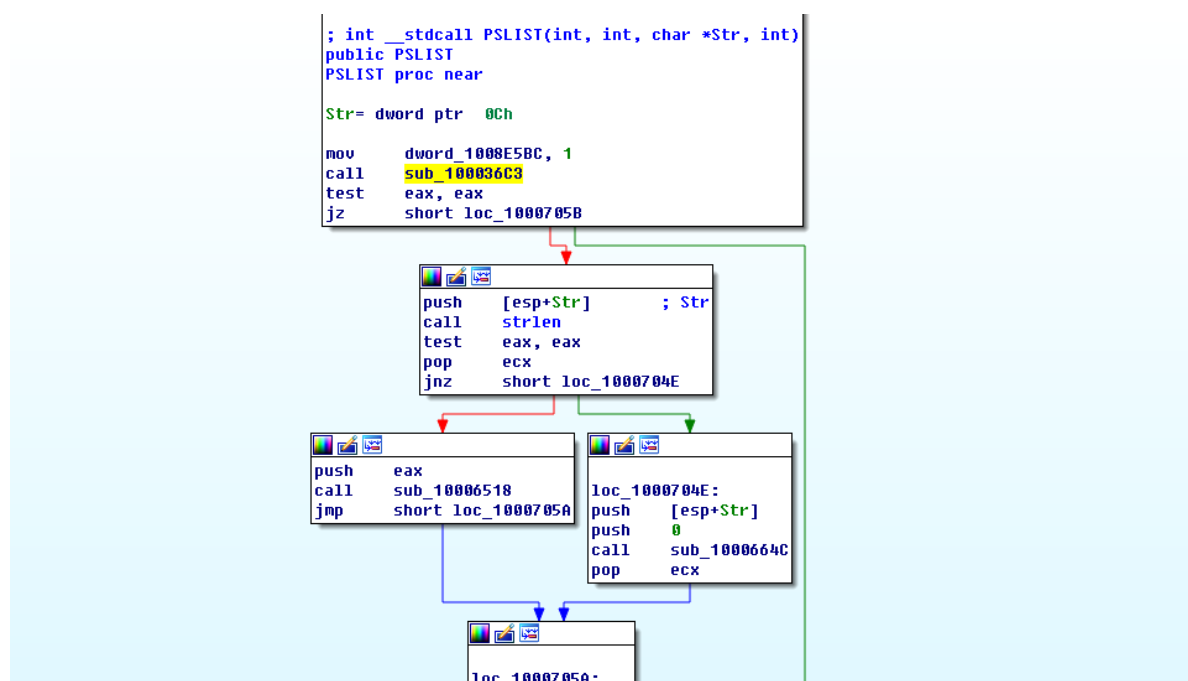
```

可以看到它查询了注册表中 HKLM\SOFTWARE\Microsoft\WindowsCurrentVersion\WorkTime 和 WorkTimes 键的值，然后将这一信息返回传给该函数的参数网络socket。

11. PSLIST导出函数做了什么？

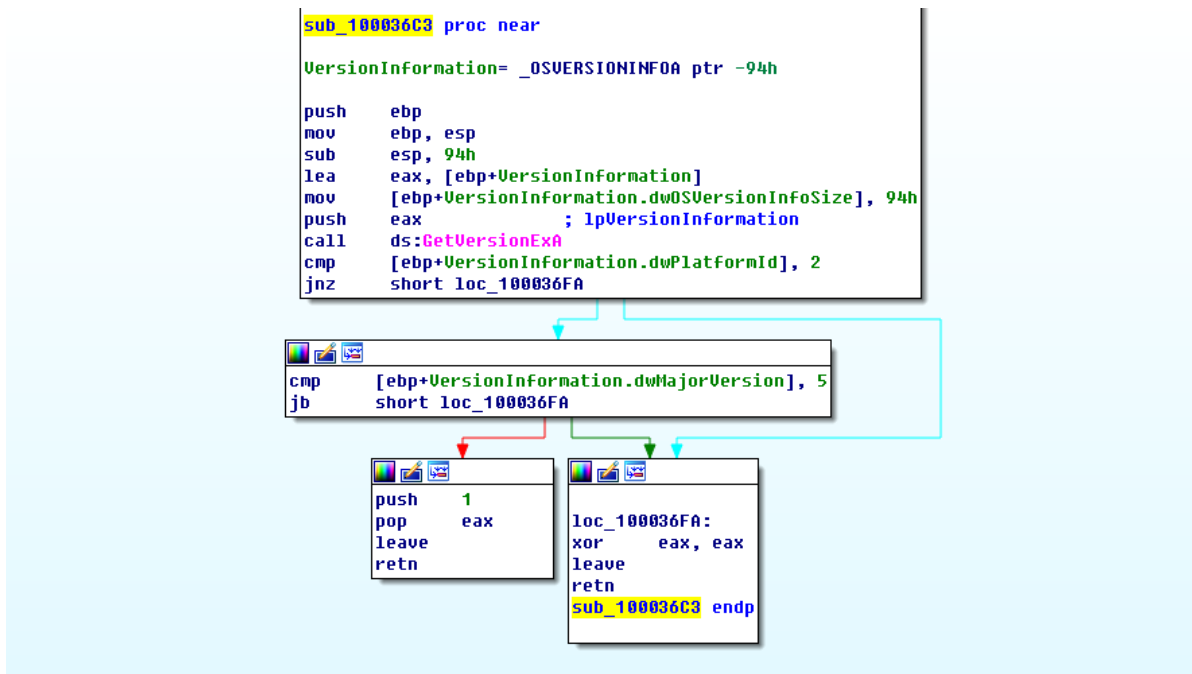
Name	Address	Ordinal
InstallRT	000000001000D847	1
InstallSA	000000001000DEC1	2
InstallSB	000000001000E892	3
PSLIST	0000000010007025	4
ServiceMain	000000001000CF30	5
StartEXS	0000000010007ECB	6
UninstallRT	000000001000F405	7
UninstallSA	000000001000EA05	8
UninstallSB	000000001000F138	9
DllEntryPoint	000000001001516D	

双击查看该函数：

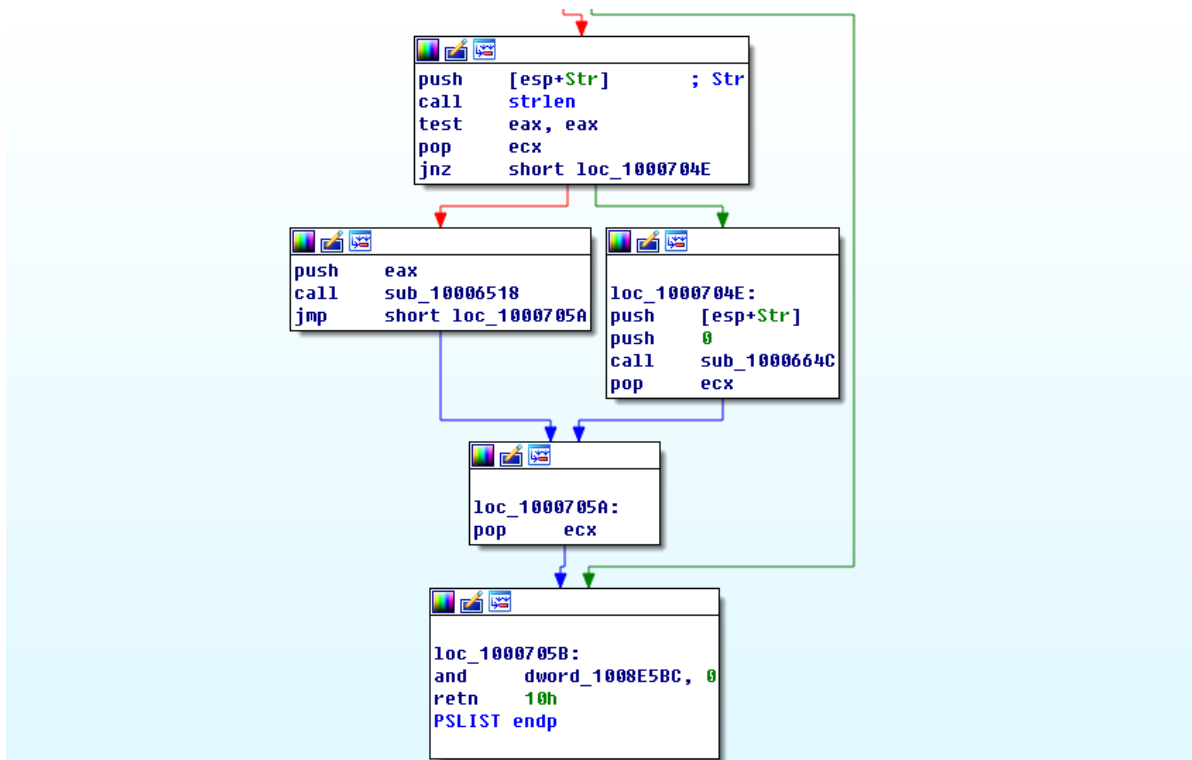


这个函数选择两条路径之一执行，这个选择取决于sub_100036C3的结果。

查看sub_100036C3：



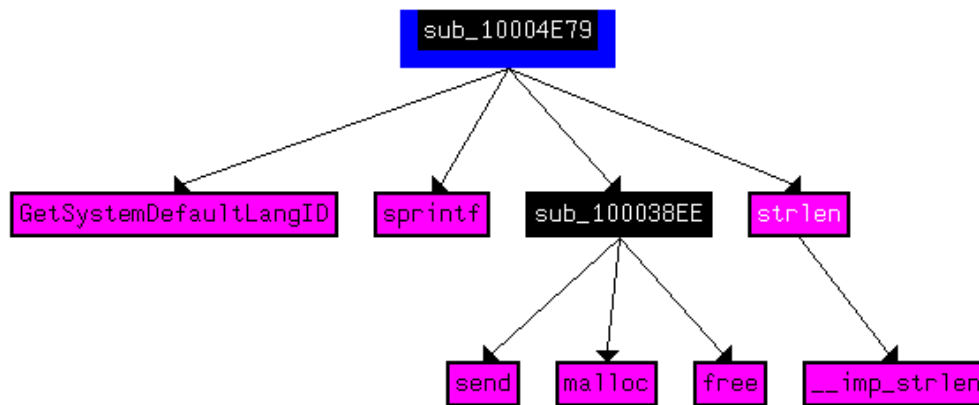
sub_100036C3函数检查操作系统的版本是Windows Vista/7,或是Windows XP/2003/2000。



这两条代码路径都是用 `CreateToolhelp32Snapshot` 函数，从相关字符串和API调用来看，用于获得一个进程列表，这两条代码都通过send将进程列表通过socket发送。

12. 使用图模式来绘制出对sub_10004E79的交叉引用图。当进入这个函数时，哪个API函数可能被调用？仅仅基于这些API函数，你会如何重命名这个函数？

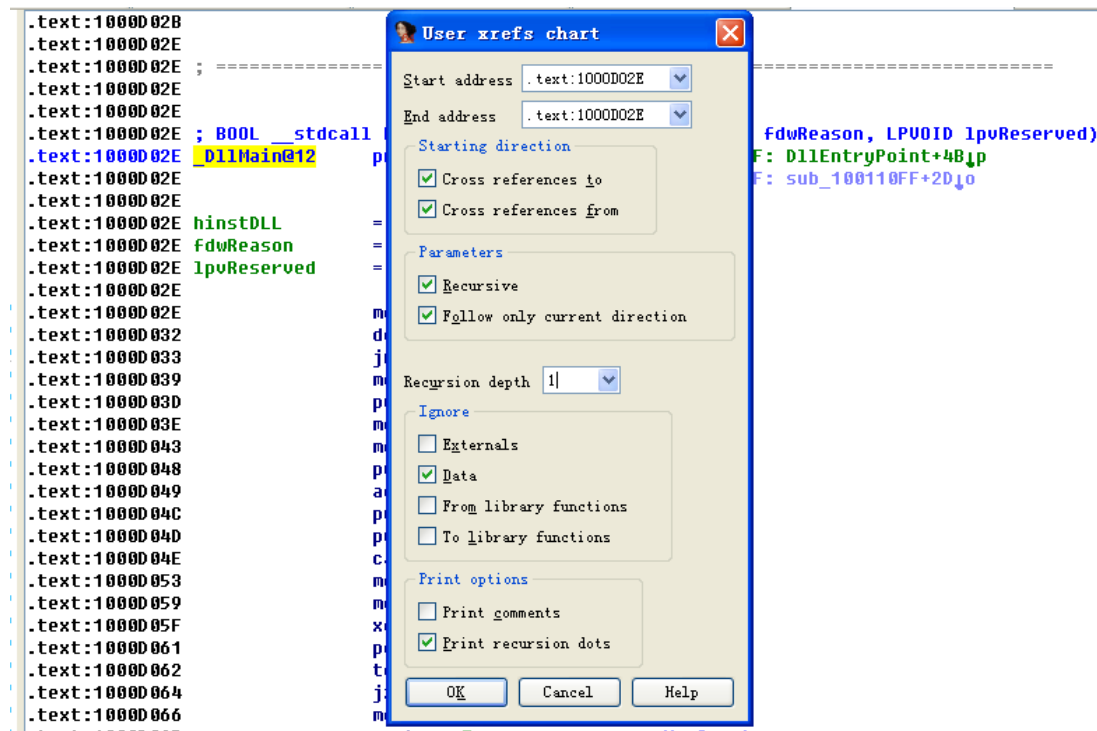
对sub_10004E79交叉引用的图示化结果：



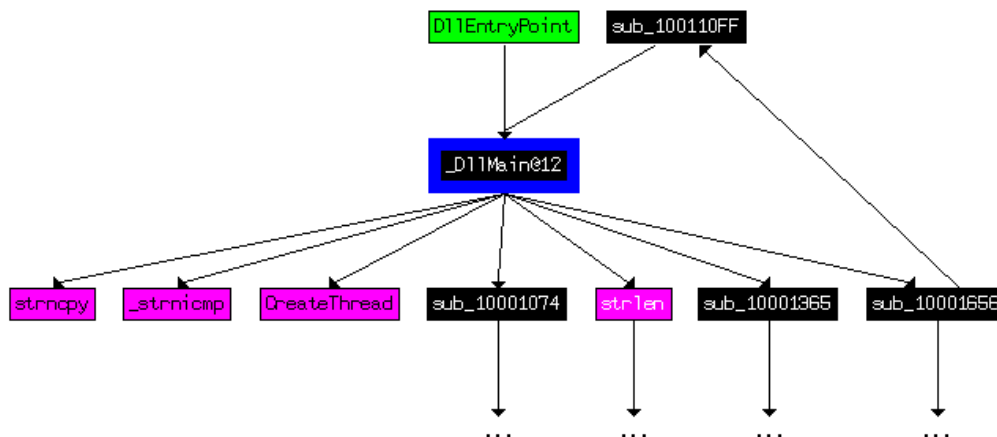
可以看到该函数调用了 `GetSystemDefaultLangID` 和 `send`。可知该函数可能通过socket发送语言标志，因此，右击函数名，因而可以直接在函数名处右键点击 `Rename` 重命名为 `send_languageID`。

13. DllMain直接调用了多少个Windows API? 多少个在深度为2时被调用?

查看直接调用的函数，设置递归深度为1



生成的交叉引用图：



直接调用的Windows API（上图中粉色函数）有四个
再将递归深度设置为2，其结果为一个非常复杂的图



大致有30多个Windows API在深度为2时被调用

14. 在0x10001358处，有一个对Sleep（一个使用包含要睡眠的毫秒数的参数的API函数）的调用。顺着代码向后看，如果这段代码执行，这个程序会睡眠多久？

查看这段代码：

```

.text:10001341 loc_10001341:                                ; CODE XREF: sub_10001074+10F7j
.text:10001341                                         ; sub_10001074+1B07j ...
.text:10001341 mov     eax, off_10019020
.text:10001346 add     eax, 00h
.text:10001349 push    eax                ; Str
.text:1000134A call    ds:atoi
.text:10001350 imul    eax, 3E8h
.text:10001356 pop     ecx
.text:10001357 push    eax                ; dwMilliseconds
.text:10001358 call    ds:Sleep
.text:1000135E xor     ebp, ebp
.text:10001360 jmp     loc_100010B4
.text:10001360 sub_10001074 endp

```

在Sleep函数调用之前，off_10019020被赋给EAX。双击查看，指向了一个字符串[This is CTI]30。

```

.data:10019020 off_10019020 dd offset aThisIsCti30 ; DATA XREF: sub_10001074:loc_10001341tr
.data:10019020                                         ; sub_10001365:loc_10001632tr ...
.data:10019020                                         ; "[This is CTI]30"

```

然后0xD被加到EAX上作为偏移，因此EAX指向了30来调用atoi，这样就把字符串30转成了数字30；

之后EAX被乘了3E8h(十进制是1000)，也就是说，对atoi调用后的返回值被乘以1000，得到要休眠的毫秒数，即30000毫秒(30秒)。

15. 在0x10001701处是一个对socket的调用。它的3个参数是什么？

跳转到地址0x10001701：

```
.text:100016FB loc_100016FB:                                ; CODE XREF: sub_10001656+374↓j
.text:100016FB                                         ; sub_10001656+A09↓j
.text:100016FB                                         ; protocol
.text:100016FD                                         ; type
.text:100016FF                                         ; af
.text:10001701                                         push    6
.text:10001701                                         push    1
.text:10001701                                         push    2
.text:10001701                                         call    ds:socket
```

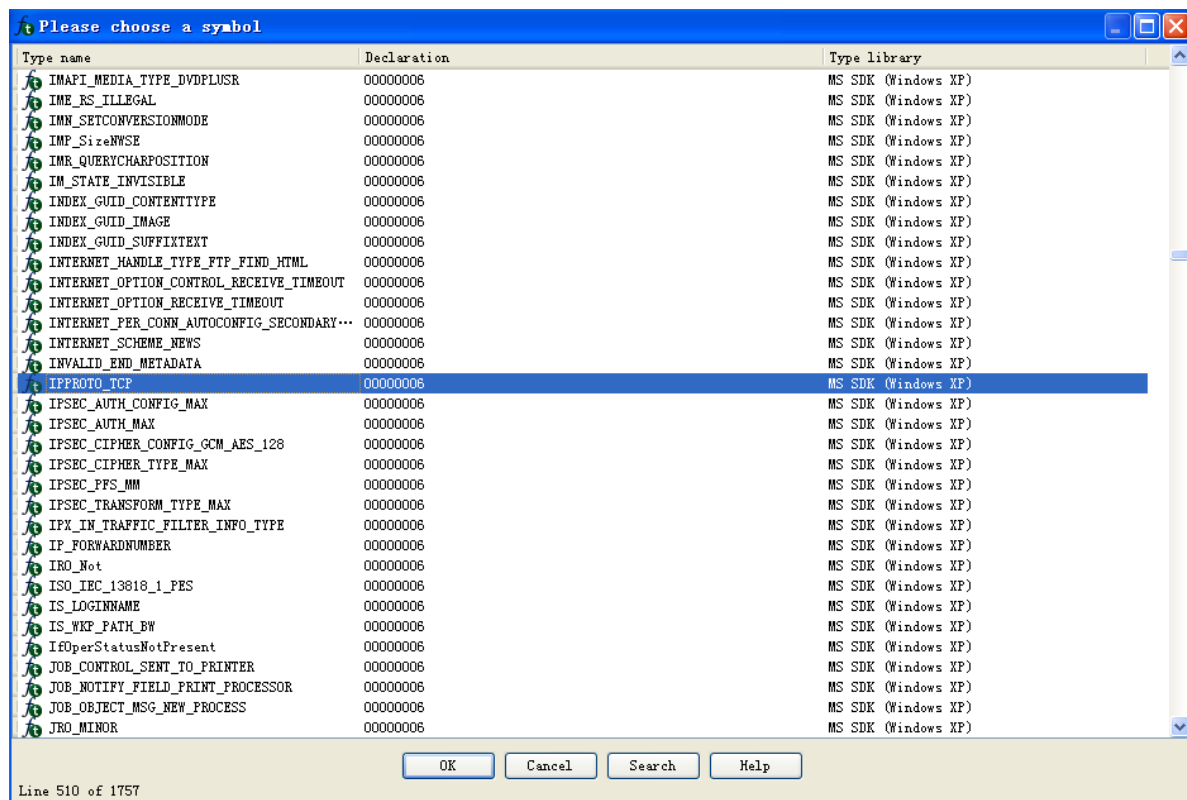
可以看到socket的三个参数6、2、1分别入栈，三个参数为af、type、protocol。

16. 使用MSDN页面的socket和IDA Pro中的命名符号常量，你能使参数更加有意义吗？在你应用了修改以后，参数是什么？

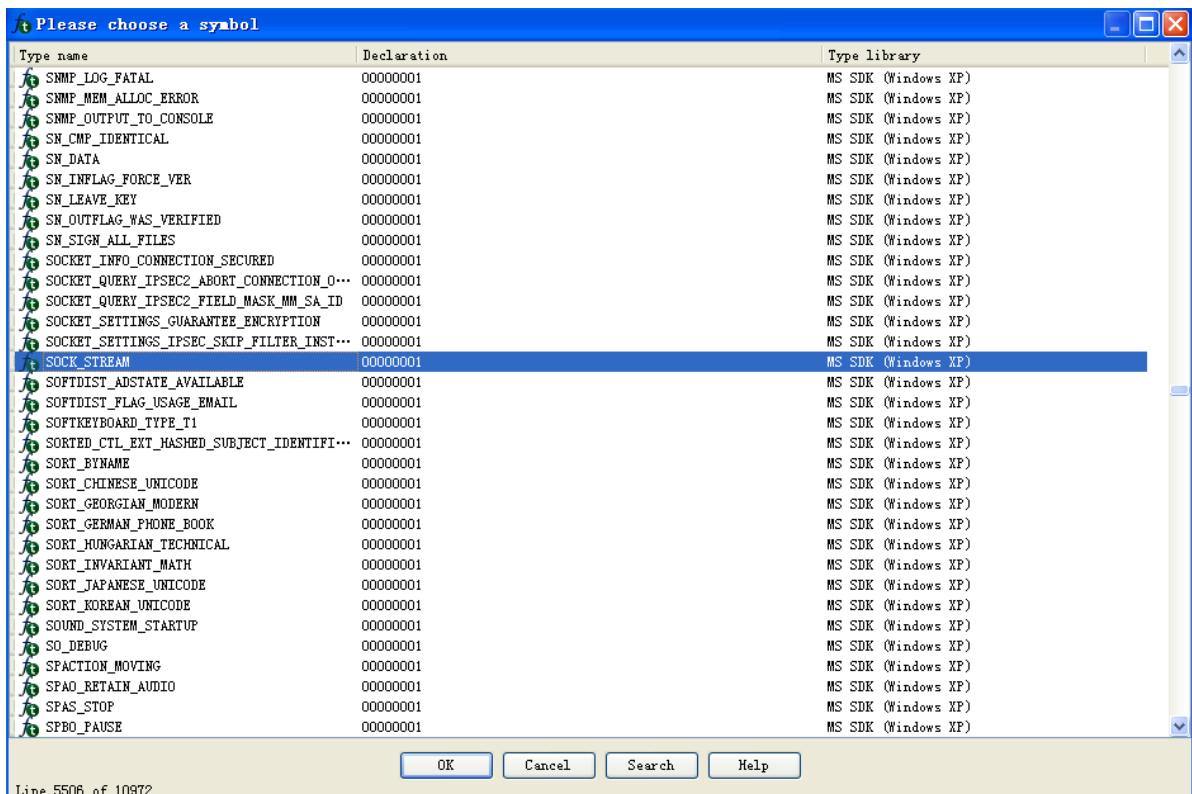
查阅socket的官方文档，这些数与MSDN页面上对socket描述中的符号常量相关。

右键单击每个数，选择Use Symbolic Constant，会弹出一个对话框，会列举出IDA Pro为这个特定值找到所有的对应常量。

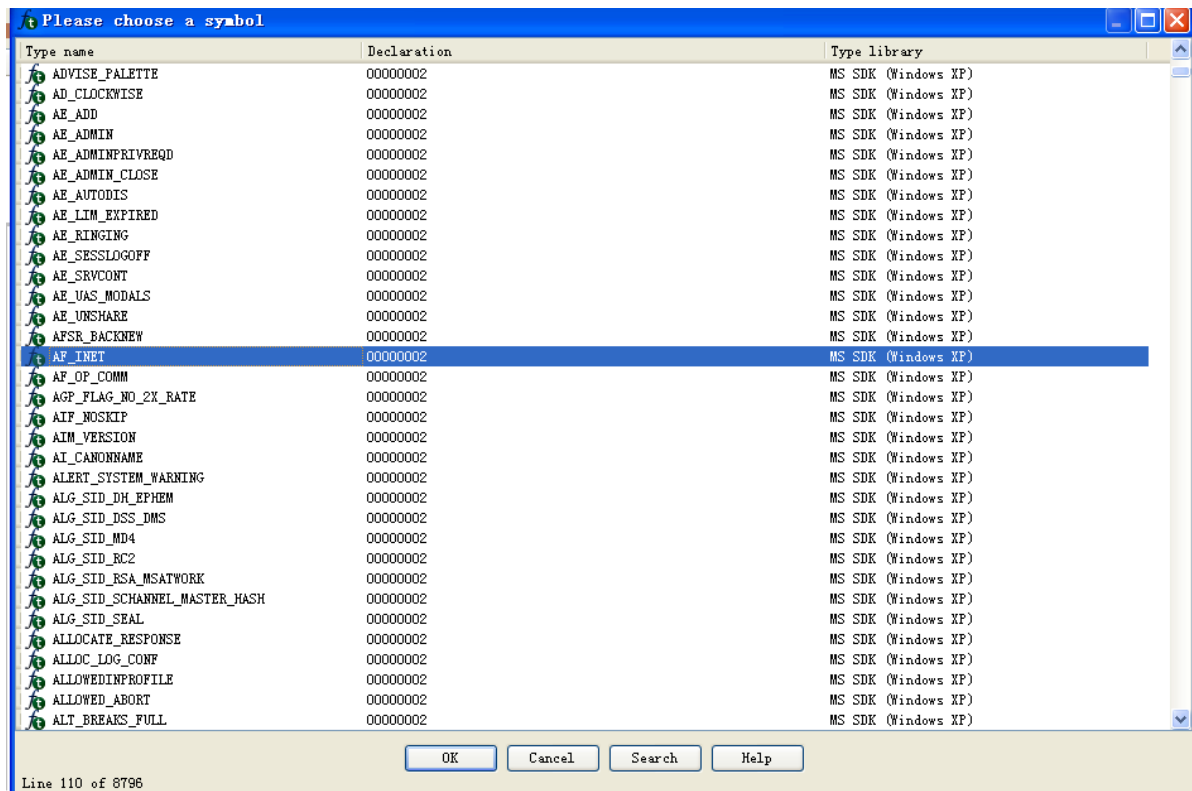
6：



1：



2:



替换后：

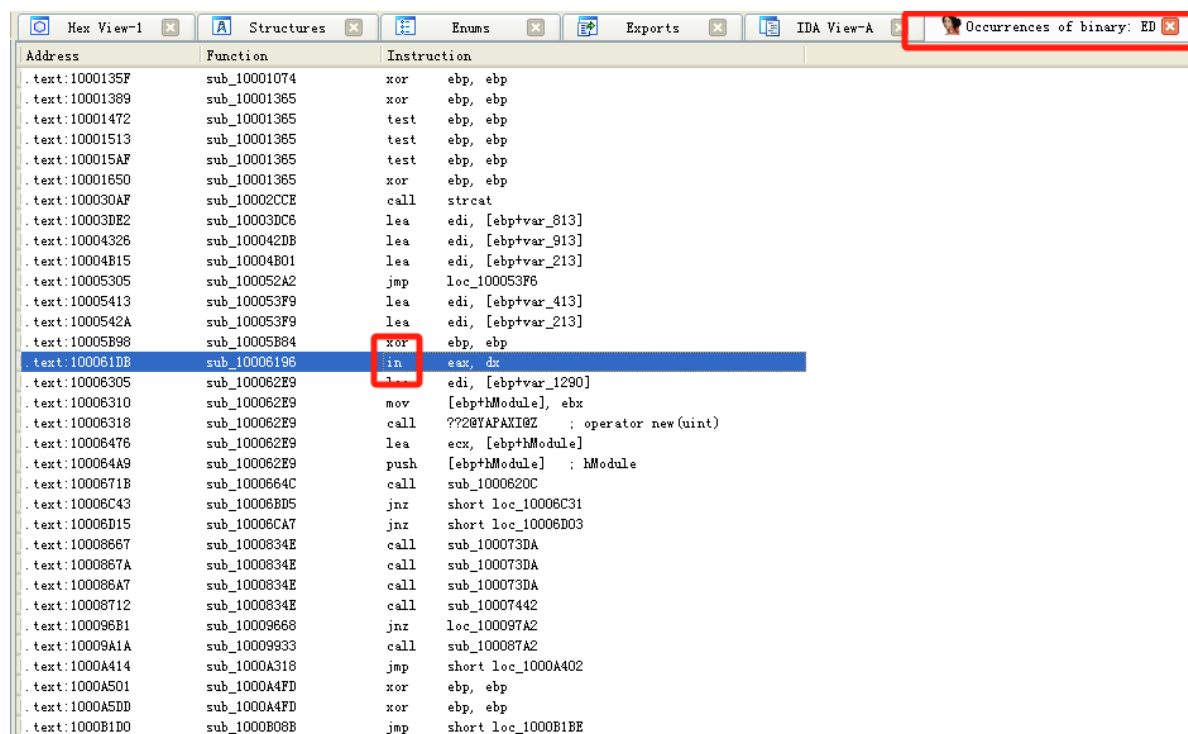
```
.text:100016FB      push     IPPROTO_TCP      ; protocol
.text:100016FD      push     SOCK_STREAM     ; type
.text:100016FF      push     AF_INET         ; af
```

值2指的是AF_INET，用于设置一个IPv4socket；值1指的是SOCK_STREAM；值6指的是IPPROTO_TCP。

因此，这个socket会被配置为基于IPv4的TCP连接(常被用于HTTP)。

17. 搜索in指令 (opcode 0xED) 的使用。这个指令和一个魔术字符串VMXh用来进行VMware检测。这在这个恶意代码中被使用了吗? 使用对执行in指令函数的交叉引用, 能发现进一步检测VMware的证据吗?

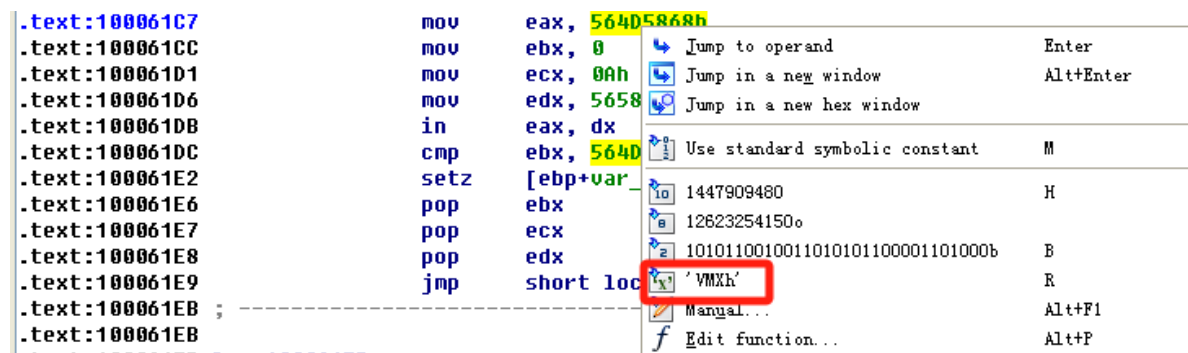
选择Search→Sequence of Bytes, 然后搜索in指令的opcode: ED, 选择Find All Occurrences, 输出一个新的窗口列出所有的匹配:



只在0x100061DB一处有in指令:

```
.text:100061C7      mov     eax, 564D5868h
.text:100061CC      mov     ebx, 0
.text:100061D1      mov     ecx, 0Ah
.text:100061D6      mov     edx, 5658h
.text:100061DB      in      eax, dx
.text:100061DC      cmp     ebx, 564D5868h
.text:100061E2      setz    [ebp+var_1C]
.text:100061E6      pop     ebx
.text:100061E7      pop     ecx
.text:100061E8      pop     edx
.text:100061E9      jmp     short loc_100061F6
```

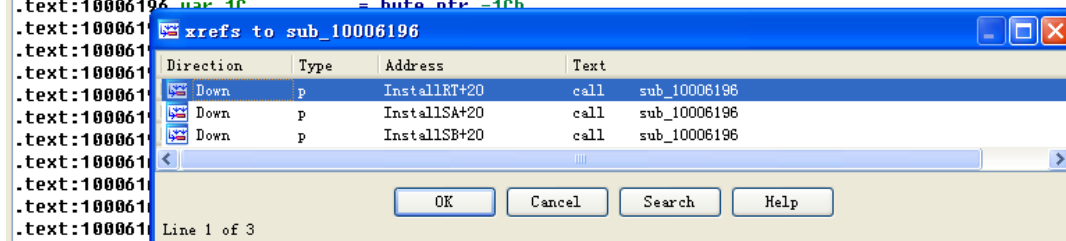
在0x100061C7处的mov指令将 564D5868h 赋给EAX。



右击这个值, 可以看到它相当于ASCII字符串 `vmxh`, 这也就确认了这段代码是恶意代码采用的反虚拟机技巧。

查看交叉引用:

```
.text:10006196 sub_10006196 proc near ; CODE XREF: InstallRT+204p
.text:10006196 ; InstallISA+204p ...
.text:10006196 var_1C = byte ptr -1Ch
```



跳转查看:

```
.text:1000D870 loc_1000D870: ; CODE XREF: InstallRT+1E7j
.text:1000D870 push offset unk_1008E5F0 ; Format
.text:1000D875 call sub_10003592
.text:1000D87A mov [esp+8+Format], offset aFoundVirtualMa ; "Found Virtual Machine,Install Cancel."
.text:1000D881 call sub_10003592
.text:1000D886 pop ecx
.text:1000D887 call sub_10005567
.text:1000D88C jmp short loc_1000D8A4
```

可以看到字符串“Found Virtual Machine,Install Cancel.”，确认其使用反虚拟机技术。

18. 将你的光标跳转到0x1001D988处，你发现了什么？

看到了一些似乎随机生成的数据字节：

```
.data:1001D988 db 2Dh ; -
.data:1001D989 db 31h ; 1
.data:1001D98A db 3Ah ; :
.data:1001D98B db 3Ah ; :
.data:1001D98C db 27h ; '
.data:1001D98D db 75h ; u
.data:1001D98E db 3Ch ; <
.data:1001D98F db 26h ; &
.data:1001D990 db 75h ; u
.data:1001D991 db 21h ; !
.data:1001D992 db 3Dh ; =
.data:1001D993 db 3Ch ; <
.data:1001D994 db 26h ; &
.data:1001D995 db 75h ; u
.data:1001D996 db 37h ; 7
.data:1001D997 db 34h ; 4
.data:1001D998 db 36h ; 6
.data:1001D999 db 3Eh ; >
.data:1001D99A db 31h ; 1
.data:1001D99B db 3Ah ; :
.data:1001D99C db 3Ah ; :
.data:1001D99D db 27h ; '
.data:1001D99E db 79h ; y
.data:1001D99F db 75h ; u
.data:1001D9A0 db 26h ; &
.data:1001D9A1 db 21h ; !
.data:1001D9A2 db 27h ; '
.data:1001D9A3 db 3Ch ; <
.data:1001D9A4 db 3Bh ; ;
.data:1001D9A5 db 32h ; 2
.data:1001D9A6 db 75h ; u
.data:1001D9A7 db 31h ; 1
.data:1001D9A8 db 30h ; 0
```

19. 如果你安装了IDA Python插件(包括IDA Pro 的商业版本的插件)，运行Lab05-01.py，一个本书中随恶意代码提供的IDA Pro Python 脚本，（确定光标是在0x1001D988处）在你运行这个脚本后发生了什么？

选择菜单File→ScriptFile，并选择Python脚本来将其运行。该脚本如下：

```
sea = ScreenEA()

for i in range(0x00,0x50):
    b = Byte(sea+i)
    decoded_byte = b ^ 0x55
    PatchByte(sea+i,decoded_byte)
```

运行结果：

• .data:1001D988	db 78h ; x
• .data:1001D989	db 64h ; d
• .data:1001D98A	db 6Fh ; o
• .data:1001D98B	db 6Fh ; o
• .data:1001D98C	db 72h ; r
• .data:1001D98D	db 20h ;
• .data:1001D98E	db 69h ; i
• .data:1001D98F	db 73h ; s
• .data:1001D990	db 20h ;
• .data:1001D991	db 74h ; t
• .data:1001D992	db 68h ; h
• .data:1001D993	db 69h ; i
• .data:1001D994	db 73h ; s
• .data:1001D995	db 20h ;
• .data:1001D996	db 62h ; b
• .data:1001D997	db 61h ; a
• .data:1001D998	db 63h ; c
• .data:1001D999	db 68h ; k
• .data:1001D99A	db 64h ; d
• .data:1001D99B	db 6Fh ; o
• .data:1001D99C	db 6Fh ; o
• .data:1001D99D	db 72h ; r
• .data:1001D99E	db 2Ch ; ,
• .data:1001D99F	db 20h ;
• .data:1001D9A0	db 73h ; s
• .data:1001D9A1	db 74h ; t
• .data:1001D9A2	db 72h ; r
• .data:1001D9A3	db 69h ; i
• .data:1001D9A4	db 6Eh ; n
• .data:1001D9A5	db 67h ; g

20. 将光标放在同一位置，你如何将这个数据转成一个单一的ASCII字符串？

保持光标在0x1001D988处，按下A键，就可以将其转换为一个ASCII字符串：

```
• .data:1001D988 aXdoorIsThisBac db 'xdoor is this backdoor, string decoded for Practical Malware Anal'
  .data:1001D988                  db 'ysis Lab :)1234',0
```

结果是 xdoor is this backdoor, string decoded for Practcal Malware Analysis Lab :)1234 。

21. 使用一个文本编辑器打开这个脚本。它是如何工作的？

使用IDLE打开

```
74 Lab05-01.py - C:\Documents and Settings\Administrator\桌面\上机实验样...
File Edit Format Run Options Windows Help
sea = ScreenEA()

for i in range(0x00,0x50):
    b = Byte(sea+i)
    decoded_byte = b ^ 0x55
    PatchByte(sea+i,decoded_byte)
```

该脚本获得光标的当前位置，用作要解码数据的偏移值。接下来从0~0x50循环：通过调用Byte获得每个字节的值，将这些字节与0x55进行XOR操作；最后将这些字节写回IDAPro的显示中，而不会去修改原始文件。

四、Yara规则

根据上面进行的分析

第八题中的 `Hi,Master [%d/%d/%d %d:%d:%d]`，17题中 `Found Virtual Machine,Install Cancel.`，第10题中有关注册表 `HKLM\SOFTWARE\Microsoft\WindowsCurrentVersion\workTime`。

使用strings工具查看更多字符串：

```
socket() GetLastError reports %d
Plug_KeyLog_Restart
xkey.dll
WSAStartup() error: %d
wb+
```

```
System
\Device\Video0
HARDWARE\DEVICEMAP\VIDEO
F333
kernel32.dll
GetDiskFreeSpaceExA
~MHz
HARDWARE\DESCRIPTION\System\CentralProcessor\0
```

根据以上特征编写yara规则：

```

rule lab5
{
strings:
    $string1 = "socket() GetLastError reports %d"
    $string2 = "WSAStartup() error: %d"
    $string3 = "HARDWARE\\DESCRIPTION\\System\\CentralProcessor\\0"
    $string4 = "SOFTWARE\\Microsoft\\Windows\\CurrentVersion"
    $string5 = "Found Virtual Machine,Install Cancel."
condition:
    filesize < 150KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
0x00004550 and all of them
}

```

使用该yara规则成功检出恶意代码：

```

PS D:\大三上\大三上\恶意代码分析与防治技术\Practical Malware Analysis Labs\Practical Malware Analysis Labs\BinaryCollection\Chapter_5
L> .\yara64.exe .\Lab5.yar .\Lab05-01.dll
lab5 .\Lab05-01.dll

```

五、IDA python脚本编写

遍历所有函数，排除库函数或简单跳转函数，当反汇编的助记符为call或者jmp且操作数为寄存器类型时，输出该行反汇编指令。

```

import idautils
for func in idautils.Functions():
    flags = idc.GetFunctionFlags(func)
    if flags & FUNC_LIB or flags & FUNC_THUNK:
        continue
    dism_addr = list(idautils.FuncItems(func))
    for line in dism_addr:
        m = idc.GetMnem(line)
        if m == 'call' or m == 'jmp':
            op = idc.GetOpType(line,0)
            if op == o_reg:
                print '0x%x %s' % (line,idc.GetDisasm(line))

```

运行脚本得到以下输出：

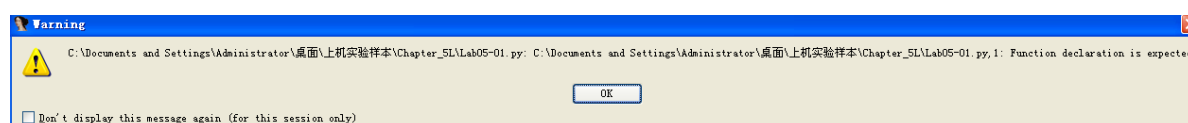
Output window		
0x1000104c	call	esi ; _stricmp
0x10001061	call	esi ; _stricmp
0x10001100	call	esi ; strchr
0x1000110f	call	esi ; strchr
0x1000111e	call	esi ; strchr
0x1000112d	call	esi ; strchr
0x10001154	call	esi ; strchr
0x10001166	call	esi ; strchr
0x1000117b	call	esi ; strchr
0x100011f1	call	esi ; strchr
0x10001203	call	esi ; strchr
0x1000121c	call	esi ; strchr
0x10001289	call	esi ; strchr
0x1000129b	call	esi ; strchr
0x100012b8	call	esi ; strchr
0x100013f1	call	esi ; strchr
0x10001400	call	esi ; strchr
0x1000140f	call	esi ; strchr
0x1000141e	call	esi ; strchr
0x10001445	call	esi ; strchr
0x10001457	call	esi ; strchr
0x1000146c	call	esi ; strchr
0x100014e2	call	esi ; strchr
0x100014f4	call	esi ; strchr
0x1000150d	call	esi ; strchr
0x1000157a	call	esi ; strchr
0x1000158c	call	esi ; strchr
0x100015a9	call	esi ; strchr
0x100016f3	call	eax
0x1000179b	call	esi ; strncpy
0x100017ad	call	esi ; strncpy
0x100017d6	call	esi ; strncpy

用于识别间接调用（通过寄存器）的函数调用或跳转，因为这类跳转通常涉及动态调度，可能会指向难以静态确定的目标。

六、实验结论及心得体会

1. 实验过程中遇到的部分问题

实验中进行到第19题时，发现IDAPro没有办法读取py文件，不能运行python脚本



尝试通过[ida中python脚本如何用 | PingCode智库](#)安装IDApthon插件解决该问题

但是发现IDA文件夹下对应文件内容都存在，但是依旧不能读取py文件

后查看IDAPro版本为3.6，查看文件夹中还有3.8版本的IDAPro，重新解压缩安装，发现在这次安装过程中就已经自动安装了IDAPython（含Python2.7），最终在IDAPro3.8下成功完成实验。

2. 心得体会

1. 在实验过程中更加熟悉了IDAPro的各类操作技巧，例如搜索技巧：可以使用G直接搜索指定地址，可以在旁边函数表中查找指定函数名，可以通过菜单栏中的search查找指定文本，或者查找指令字节查询。同时我也发现，使用十六进制进行字节查找的速度是比查找对应文本效率更高，准确性也更高的。
2. 同时也接触使用了例如绘制交叉引用图等IDAPro的功能性使用，对之后分析代码有很大帮助。

3. 在本次理论课上也学习了IDA python的编写内容，并在实验过程中尝试运行编写脚本得到特定指令，从而可以获取常用于识别间接调用（通过寄存器）的函数调用或跳转等需要特别关注的指令。对之后编写IDA python脚本有了一定基础。